

MUSHARAF AIJAZ BABA

1. Create a new storage account and a new container (Musharaf-raw)

[Home](#) > [Storage accounts](#) >

Create a storage account



Basics

Advanced

Networking

Data protection

Encryption

Review + create



[View automation template](#)

Basics

Subscription	LabsKraft
Resource group	LabsKraft2027
Location	Qatar Central
Storage account name	musharafcapstone2
Primary service	
Performance	Standard
Replication	Locally-redundant storage (LRS)

Advanced

Enable hierarchical namespace	Enabled
Enable SFTP	Disabled
Enable network file system v3	Disabled
Allow cross-tenant replication	Disabled
Access tier	Hot

Previous

Next

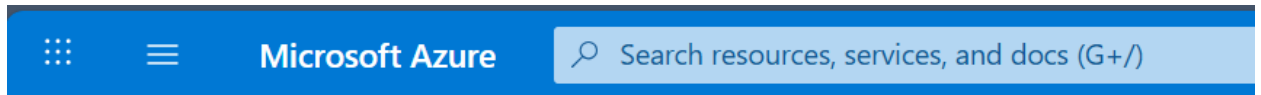
Create

[Give feedback](#)

2. Upload your daily sales CSV files to the raw/sales/ folder. Upload your product metadata (JSON/CSV) files to the raw/products/ folder.

[Type here]

3. Now create a new Azure Connector



[Home](#) > [Access Connector for Azure Databricks](#) >

Create an Access Connector for Azure Databricks

[Basics](#) [Tags](#) [Managed Identity](#) [Review + create](#)

 [View automation template](#)

Basics

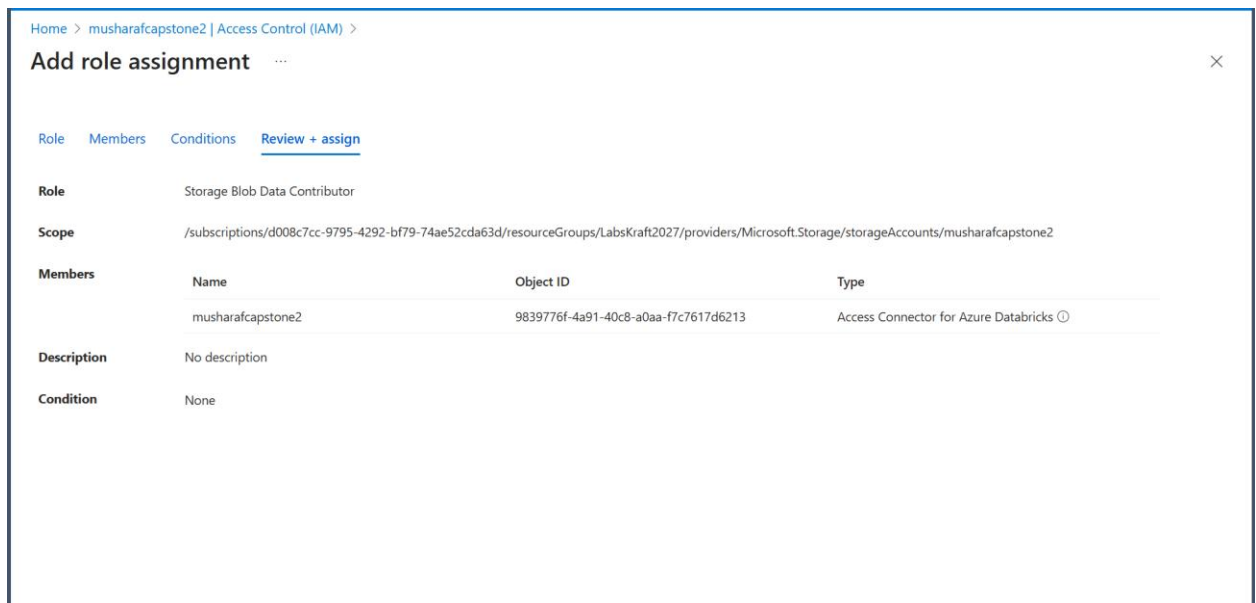
Subscription	LabsKraft
Resource group	LabsKraft2027
Name	musharafcapstone2
Region	Qatar Central

Managed Identity

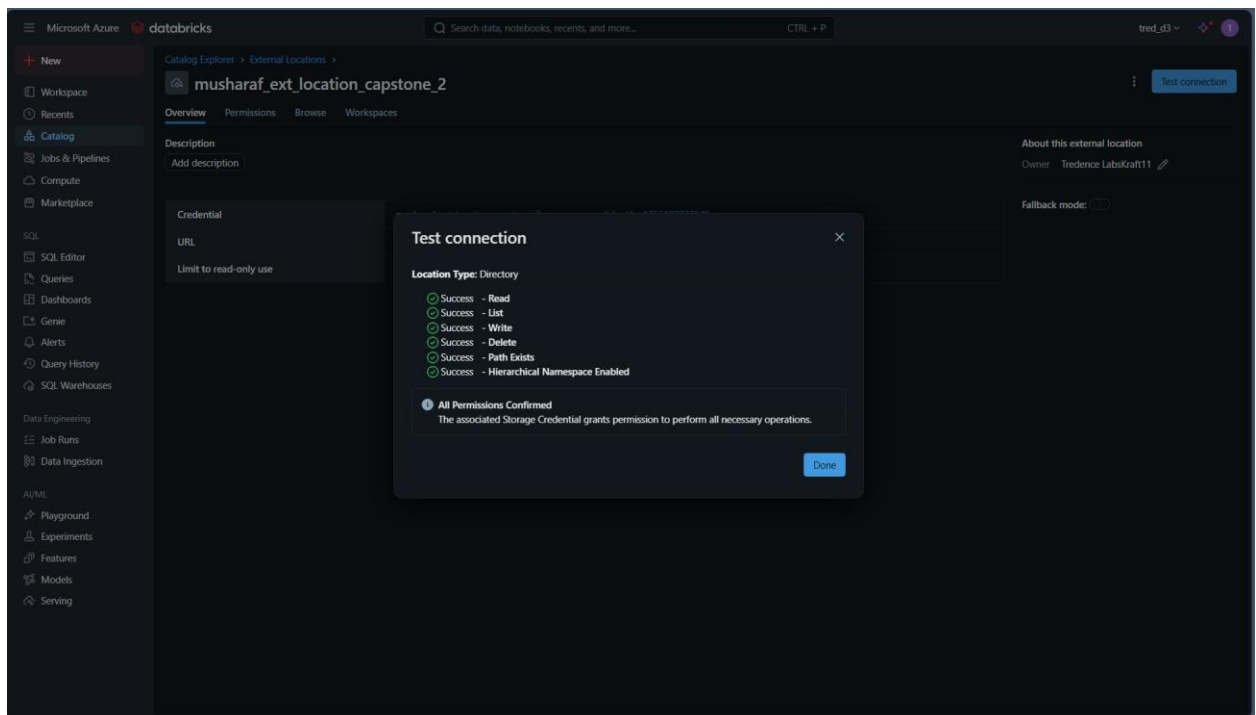
Identity type	SystemAssigned
---------------	----------------

[Type here]

4. After creating a new Access Connector, add IAM roles to the ADLS Blob storage

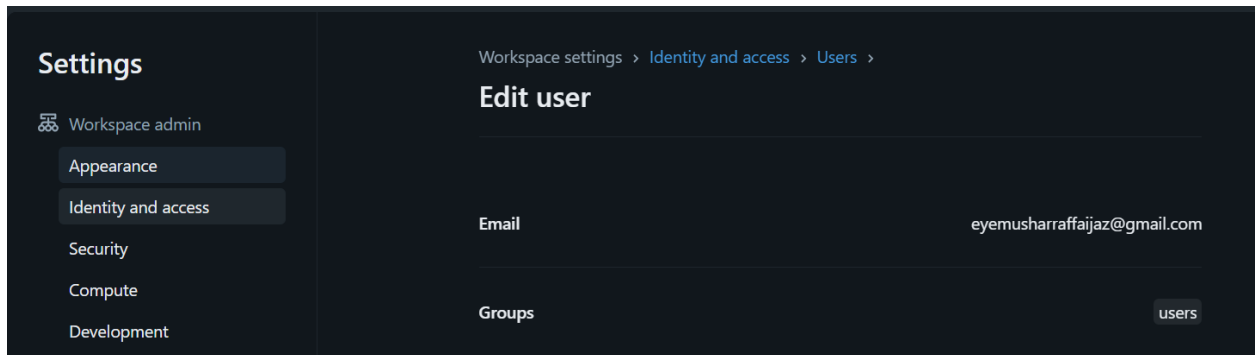


5. Now in the Databricks, create a new catalogue



[Type here]

6. After creating a new catalogue, go to settings > identity & Access > Add User and add a new user (eyemusharraffaijaz@gmail.com)



7. Now Create a new Scheme, using the following command:

```
\ %sql
```

```
CREATE CATALOG IF NOT EXISTS musharaf_capstone_2;
```

```
USE CATALOG musharaf_capstone_2;
```

```
CREATE SCHEMA IF NOT EXISTS musharaf_capstone_schema;
```

```
USE SCHEMA musharaf_capstone_schema;
```

```
\
```

8. Now create a DTL Pipeline:

Query history

Statement	Started At	Duration	Rows read	Bytes read	Bytes written
REFRESH MATERIALIZED VIEW musharaf_capstone_2.musharaf_c	Aug 26, 2025, 11:43 AM	11 s 416 ms	9,002	98.99 KB	0 B
REFRESH MATERIALIZED VIEW musharaf_capstone_2.musharaf_c	Aug 26, 2025, 11:42 AM	12 s 692 ms	9,002	128.89 KB	49.50 KB
REFRESH STREAMING TABLE musharaf_capstone_2.musharaf_ca	Aug 26, 2025, 11:42 AM	15 s 674 ms	5,001	199.13 KB	0 B
REFRESH STREAMING TABLE musharaf_capstone_2.musharaf_ca	Aug 26, 2025, 11:42 AM	24 s 36 ms	1,001	211.59 KB	18.72 KB
REFRESH STREAMING TABLE musharaf_capstone_2.musharaf_ca	Aug 26, 2025, 11:42 AM	25 s 321 ms	5,000	252.49 KB	99.57 KB
REFRESH STREAMING TABLE musharaf_capstone_2.musharaf_ca	Aug 26, 2025, 11:42 AM	24 s 701 ms	8,001	174.89 KB	45.72 KB

musharaf_capstone_2_Pipeline

Run: 8/26/2025, 11:41:04 AM - Completed

Select tables for refresh

Graph | List

Streaming table: products_raw_csv, products_raw_json, sales_raw

View: products_raw_unifi...

Materialized view: products_clean, sales_daily_summa...

Event log | Query history

All | Info | Warning | Error

level:error

[Type here]

```

import dlt
from pyspark.sql.functions import *

# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
    name="sales_raw",
    comment="Raw sales data ingested from CSV files",
    table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles") # Auto Loader for incremental loads
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/sales/")
    )

# Products CSV
@dlt.table(
    name="products_raw_csv",
    comment="Raw product data from CSV",
    table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/products/")
    )

# Products JSON
@dlt.table(
    name="products_raw_json",
    comment="Raw product data from JSON",
    table_properties={"quality": "bronze"}
)

```

[Type here]

```

)
def products_raw_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("abfss://musharaf-
raw@musharafcapstone2.dfs.core.windows.net/products/")
    )

# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
    return dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"),
allowMissingColumns=True)

# Clean Sales
@dlt.table(
    name="sales_clean_table",
    comment="Cleaned sales data with enforced schema",
    table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean_table():
    return (
        dlt.read("sales_raw")
        .withColumn("sale_id", col("sale_id").cast("string"))
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("region", col("region").cast("string"))
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )

@dlt.table(
    name="products_clean",

```

[Type here]

```

    comment="Cleaned products data with schema evolution support",
    table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read("products_raw")
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("product_name", col("product_name").cast("string"))
        .withColumn("category", col("category").cast("string"))
        .withColumn("price", col("price").cast("double"))
        .withColumn("discount_rate", col("discount_rate").cast("double"))
    )

# Join sales + products and calculate KPIs
@dlt.table(
    name="sales_summary",
    comment="Daily sales summary by product and region",
    table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean_table")
    products = dlt.read("products_clean")
    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name", "category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )

```

[Type here]

9. Once this is done, now change the schema or introduce corruption in the data files (ex csv in sales), then the pipelines should show error:

The screenshot shows the Databricks Jobs & Pipelines interface for a pipeline named 'musharaf_capstone_2_Pipeline'. The pipeline is in a 'Failed' state, with a run from 8/26/2025, 2:23:48 PM. The pipeline graph shows several stages: 'products_raw_csv' (Streaming table, Completed: 22s), 'products_raw_json' (Streaming table, Completed: 11s), 'products_raw' (View), 'products_clean' (Materialized view, Completed: 22s), 'sales_raw' (Streaming table, Completed: 11s), 'sales_clean_table' (Materialized view, Failed: Show latest error), and 'sales_summary' (Materialized view, Staged). The 'Event log' at the bottom shows two error messages: 'Flow 'musharaf_capstone_2.musharaf_capstone_schema.sales_clean_table' has FAILED.' and 'Update 05b918 is FAILED since flow 'musharaf_capstone_2.musharaf_capstone_schema.sales_clean_table' has failed.'

10. Code after fix:

```
import dlt
from pyspark.sql.functions import *
from pyspark.sql.types import *

# Configuration for schema locations - these are still needed to track schema
# evolution
sales_schema_location = "abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/schema/sales/"
products_csv_schema_location = "abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/schema/products_csv/"
products_json_schema_location = "abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/schema/products_json/"

# Note: We no longer define explicit schemas here.
# The pipeline will now infer the schema from the data itself.

# Ingest raw sales data with automated schema evolution
@dlt.table(
    name="sales_raw",
    comment="Raw sales data ingested from CSV files with automated schema inference",
    table_properties={"quality": "bronze"}
)
```

[Type here]


```

def sales_raw():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.schemaLocation", sales_schema_location)
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns") #
Automatically adds new columns
        .load("abfss://musharaf-
raw@musharafcapstone2.dfs.core.windows.net/sales/")
    )

# Products CSV with automated schema evolution
@dlt.table(
    name="products_raw_csv",
    comment="Raw product data from CSV with automated schema inference",
    table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.schemaLocation", products_csv_schema_location)
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("abfss://musharaf-
raw@musharafcapstone2.dfs.core.windows.net/products/")
    )

# Products JSON with automated schema evolution
@dlt.table(
    name="products_raw_json",
    comment="Raw product data from JSON with automated schema inference",
    table_properties={"quality": "bronze"}
)
def products_raw_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("multiLine", "true") # Crucial for multi-line JSON files
        .option("cloudFiles.schemaLocation", products_json_schema_location)
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    )

```

[Type here]

```

        .load("abfss://musharaf-raw@musharafcapstone2.dfs.core.windows.net/products/")
    )

# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
    return
dlt.read_stream("products_raw_csv").unionByName(dlt.read_stream("products_raw_json"), allowMissingColumns=True)

# Clean Sales with data quality rules. We rely on inferred schema.
@dlt.table(
    name="sales_clean_table",
    comment="Cleaned sales data with data quality checks",
    table_properties={"quality": "silver"}
)
@dlt.expect_or_drop("valid_sales_amount", "sales_amount >= 0")
@dlt.expect_or_drop("valid_quantity", "quantity > 0")
def sales_clean_table():
    # We now read the stream directly and apply transformations.
    # The schema is already inferred from the source.
    return (
        dlt.read_stream("sales_raw")
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )

# Clean Products. We rely on inferred schema and coalesce on discount_rate.
@dlt.table(
    name="products_clean_table",
    comment="Cleaned products data with schema evolution support",
    table_properties={"quality": "silver"}
)
def products_clean_table():
    return (
        dlt.read_stream("products_raw")
        .withColumn("discount_rate", coalesce(col("discount_rate"), lit(0.0)))
    )

# Join sales + products and calculate KPIs
@dlt.table(
    name="sales_summary",

```

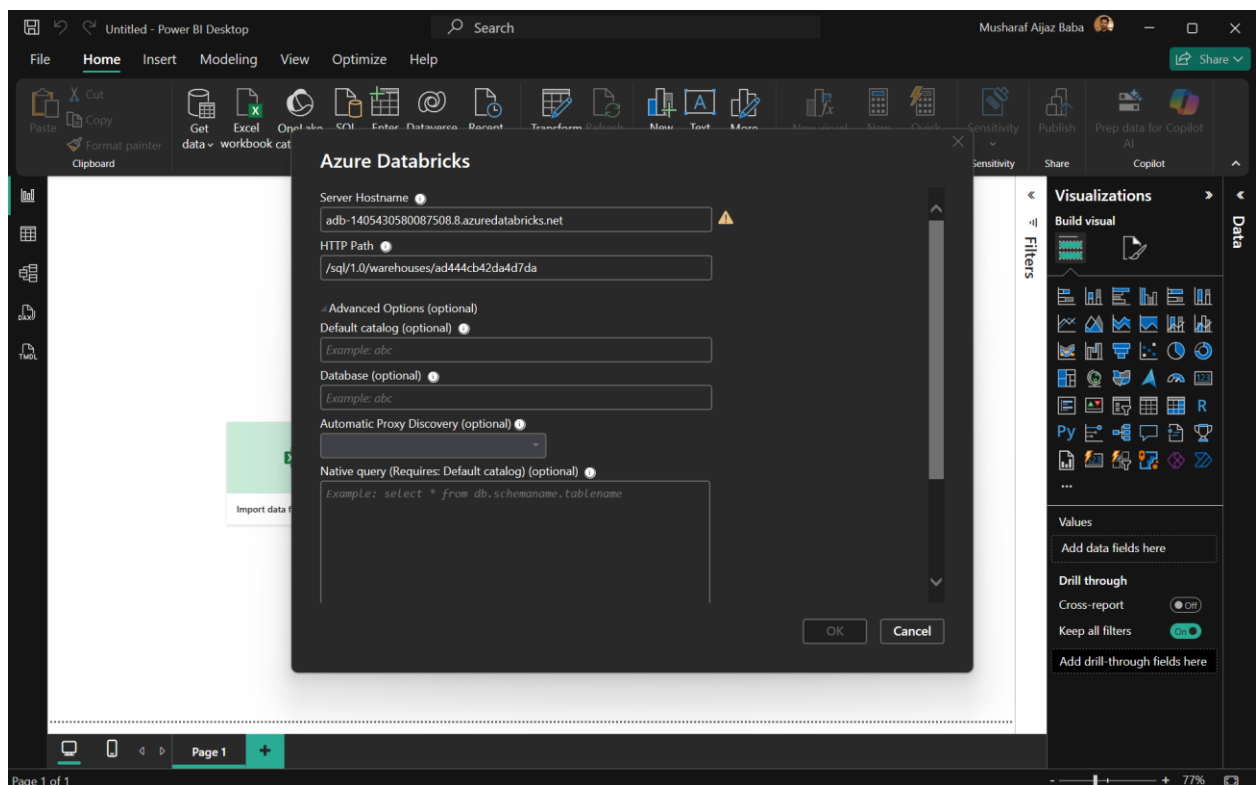
[Type here]

```

    comment="Daily sales summary by product and region",
    table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read_stream("sales_clean_table")
    products = dlt.read("products_clean_table")
    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name", "category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
)

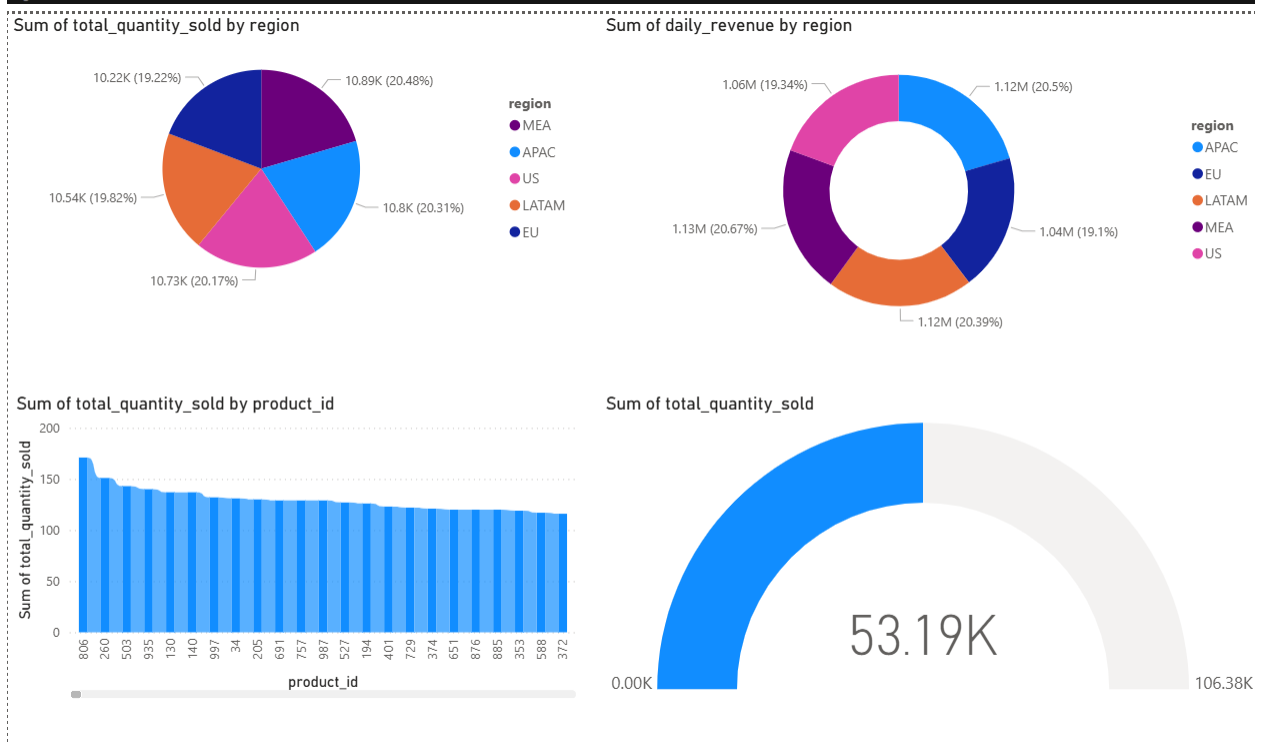
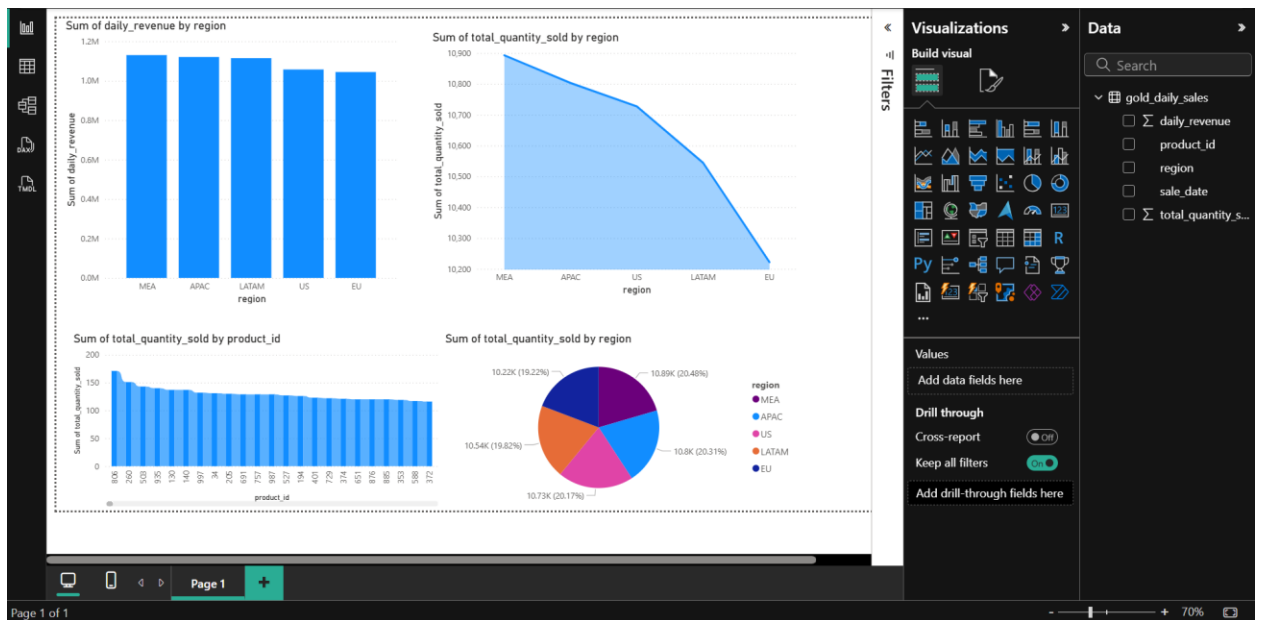
```

11. Now we will visualize this in Power Bi, open Power Bi and connect the Azure DataBricks



[Type here]

12. Now select the values and add visualizations to the data



[Type here]